

RETRO VINTAGE QUIZ ARCADE

Complete End-to-End System Architecture, Serverless Backend & Cloud RDBMS Engineering Guide

■ SYSTEM OVERVIEW

A full-stack, authentic 1980s/1990s arcade quiz platform running on **Next.js 14 App Router**, synthesized **Web Audio API** sound synthesizer, and **Neon Serverless PostgreSQL** relational database cloud.

≡ CORE HIGHLIGHTS

- **Cloud RDBMS:** Neon PostgreSQL Serverless
- **Deployment:** 1-Click Vercel Edge Serverless
- **State Engine:** Zero-Race Sync Ref Tracker
- **Palette:** 4-Color High Contrast Standard
- **Audio:** Procedural 8-bit Sound (0 WAVs)

🛡️ DATA INTEGRITY

Full relational schema with foreign key cascades, self-healing auto-migrations, automatic seeders, and protected 32-bit integer boundaries.

📖 REPOSITORY & LIVE URL

GitHub: [RAFIRE-ESSE/retro-quiz](#)

Live: [retro-quiz-ivory.vercel.app](#)

1. SYSTEM ARCHITECTURE & SYMBOLS

Custom Retro Theme Symbols Dictionary

The system utilizes an iconic retro arcade symbolic convention across the UI and documentation:

 ARCADE_CORE (Game Engine)

 CARTRIDGE (Quiz Package)

 RDBMS_CLOUD (Neon PostgreSQL)

 BLITZ_CLOCK (Time Attack)

 PERMADEATH (Sudden Death)

 HALL_OF_FAME (Leaderboard)

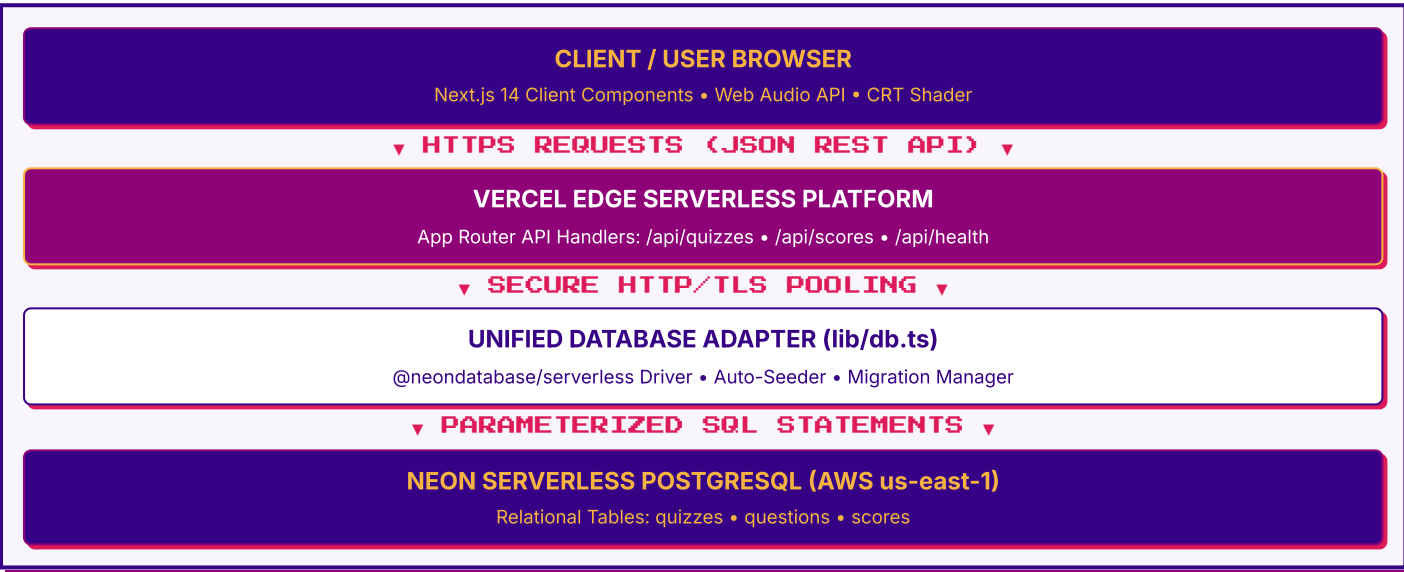
 INSERT_COIN (Player Auth)

 ACID_TX (Transaction Guard)

 EDGE_DISPATCH (Vercel Serverless)

 HISTORICAL_LOG (Question Rationale)

High-Level System Flow Diagram



Strict 4-Color Arcade Design System

To ensure total visual consistency, every screen is rendered in four distinct vintage tones:

#360185
DEEP INDIGO

#8F0177
BORDEAUX PLUM

#DE1A58
CRIMSON ROSE

#F4B342
VINTAGE GOLD

⚡ 2. REAL-TIME GAME & SCORING ENGINE

Deterministic Scoring Algorithm

$$\text{EARNED_POINTS} = \text{Math.round}((100 + \text{SPEED_BONUS}) * \text{STREAK_MULTIPLIER})$$

Component	Calculation Rule	Game Mode Application
Base Points	+100 PTS per correct answer	All Modes (Standard, Blitz, Practice, Sudden Death)
Speed Bonus	Remaining Seconds × 10 PTS	Standard (15s clock) & Sudden Death (15s clock)
Blitz Bonus	Flat +30 PTS (+3s added to global clock)	Time Attack Blitz (60s global clock)
Mistake Penalty	0 PTS, Streak resets to 0 (-2s in Blitz)	Standard, Blitz, Practice. (Sudden Death: Game Over)

Streak Multipliers (Arcade Combo Scale)

Streak Count	Multiplier	HUD Badge & Sound Effect
1 Question	1.0× (Base)	Normal Chime
2 – 3 Questions	1.2× (+20%)	COMBO x1.2!
4 – 5 Questions	1.5× (+50%)	MEGA COMBO x1.5!
6+ Questions	2.0× (+100% Double)	ULTRA STREAK x2.0! + High Tone Fanfare

State Synchronization & Closure Prevention Architecture

In standard React state updates, calling asynchronous delays like `setTimeout` on the final question captures stale closure states, causing lost points on the results screen. The engine enforces **Synchronous Mutable References**:

```
// Synchronous Engine State in app/quiz/[id]/page.tsx
const scoreRef = useRef(0);
const streakRef = useRef(0);
const maxStreakRef = useRef(0);
const answersLogRef = useRef<AnswerLogItem[]>([]);
const totalTimeSpentRef = useRef(0);

// Answer Evaluation Execution (Synchronous Mutation)
const handleAnswerSelect = (optionIndex: number) => {
  if (isAnswerSubmitted) return;
  const isCorrect = optionIndex === currentQ.correctOption;
  if (isCorrect) {
    streakRef.current += 1;
    if (streakRef.current > maxStreakRef.current) maxStreakRef.current = streakRef.current;
    const earned = Math.round((100 + timeBonus) * multiplier);
    scoreRef.current += earned;
  } else {
    streakRef.current = 0;
  }
  answersLogRef.current.push(logEntry);
  setScore(scoreRef.current); // React state updated exclusively for visual HUD
};
```

3. CLOUD DATABASE & API SPECIFICATION

Neon PostgreSQL Normalized Schema

```
-- Quizzes Metadata Table
CREATE TABLE IF NOT EXISTS quizzes (
  id SERIAL PRIMARY KEY,
  slug VARCHAR(100) UNIQUE NOT NULL,
  title VARCHAR(200) NOT NULL,
  description TEXT NOT NULL,
  category VARCHAR(50) NOT NULL,
  icon VARCHAR(10) DEFAULT '🕹️',
  difficulty VARCHAR(20) DEFAULT 'medium',
  is_custom BOOLEAN DEFAULT false,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Questions Relational Child Table
CREATE TABLE IF NOT EXISTS questions (
  id SERIAL PRIMARY KEY,
  quiz_id INT NOT NULL REFERENCES quizzes(id) ON DELETE CASCADE,
  question_text TEXT NOT NULL,
  code_snippet TEXT,
  option_a TEXT NOT NULL, option_b TEXT NOT NULL, option_c TEXT NOT NULL, option_d TEXT NOT NULL,
  correct_option INT NOT NULL CHECK (correct_option BETWEEN 0 AND 3),
  explanation TEXT NOT NULL, difficulty VARCHAR(20) DEFAULT 'medium'
);

-- Leaderboard High Scores Table
CREATE TABLE IF NOT EXISTS scores (
  id SERIAL PRIMARY KEY, quiz_id INT, quiz_title VARCHAR(200), gamer_tag VARCHAR(50) NOT NULL,
  score INT NOT NULL, accuracy INT NOT NULL, max_streak INT DEFAULT 0, game_mode VARCHAR(30) DEFAULT 'standard',
  time_spent_seconds INT DEFAULT 0, played_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

Serverless REST API Route Reference

Endpoint	Method	Description	Sample Response
/api/quizzes	GET	Fetches all active cartridges	{ "quizzes": [{ "id": 1, "slug": "retro-tech", ... }] }
/api/quizzes	POST	Creates new custom cartridge	{ "success": true, "quiz": { "id": 5, "slug": "..."} }
/api/quizzes/{id}	GET	Retrieves cartridge by slug/ID	{ "quiz": { "id": 1, "questions": [...] } }
/api/scores	GET	Returns top arcade leaderboard	{ "scores": [{ "gamerTag": "TEST_PILOT", ... }] }
/api/scores	POST	Records validated player score	{ "success": true, "score": { "id": 6, "score": 1980 } }
/api/health	GET	Database connectivity inspector	{ "status": "ok", "database": { "connected": true } }

Leaderboard Query with LEFT JOIN Resilience

```
SELECT s.id, s.quiz_id AS "quizId", COALESCE(q.title, s.quiz_title, 'Retro Cartridge') AS "quizTitle",
       s.gamer_tag AS "gamerTag", s.score, s.accuracy, s.max_streak AS "maxStreak",
       s.game_mode AS "gameMode", s.time_spent_seconds AS "timeSpentSeconds", s.played_at AS "playedAt"
FROM scores s LEFT JOIN quizzes q ON s.quiz_id = q.id ORDER BY s.score DESC, s.played_at DESC LIMIT 50;
```

4. DEPLOYMENT & RUNTIME OPERATIONS

Automated CI/CD Pipeline (GitHub → Vercel → Neon)

- 1. Git Push:** Commit pushed to `https://github.com/RAFIRE-ESSE/retro-quiz.git`.
- 2. Vercel Build:** Webhook executes `npm run build`, running Next.js static tree-shaking and route compilation.
- 3. Edge Execution:** Serverless Route Handlers are provisioned in AWS us-east-1 data centers.
- 4. Database Link:** App reads `DATABASE_URL` and connects via `aneondatabase/serverless` over pooled HTTPS transport.
- 5. Self-Healing Initialization:** If database tables are fresh, `initNeonDatabase()` automatically creates all schemas and seeds the initial cartridges.

Required Environment Variables

VARIABLE	PURPOSE	ENVIRONMENT
DATABASE_URL	Neon Serverless PostgreSQL connection string	Production & Development
POSTGRES_URL	Vercel Storage Postgres alias connection URL	Production (Auto-injected)
MSSQL_SERVER	(Optional) Microsoft SQL Server host for hybrid setups	Optional

Operational Commands

```
# 1. Install Dependencies
npm install

# 2. Run Local Development Server
npm run dev

# 3. Create Production Optimized Build
npm run build

# 4. Start Production Server
npm start
```

⚠ SYSTEM HEALTH VERIFICATION

To verify live database connectivity at any time, perform a health check:

```
curl -s https://retro-quiz-ivory.vercel.app/api/health
```

Expected status: `{"status":"ok","database":{"connected":true,"mode":"neon_postgres"}}`